CS 256 – Programming Languages & Translators
Homework Assignment #5

- This assignment is **due by 11 a.m. on Monday, October 22nd.**
- This assignment will be worth **5%** of your course grade.
- You are to work on this assignment **by yourself**.
- You are encouraged to **take a look at the sample input and output files** posted on the Blackboard website **before** you actually submit your assignment for grading.

Basic Instructions

For this assignment you are to finish developing the MFPL interpreter by modifying your HW #4 program to make it also **evaluate expressions**.

As before, your program **must** compile and execute on one of the campus Linux machines (such as rc*nn*ucs213.managed.mst.edu where ***nn*** is **01-08**). If your *flex* file was named **mfpl.l** and your *bison* file was named **mfpl.y**, we should be able to compile and execute them using the following commands (where *inputFileName* is the name of some input file):

```
flex mfpl.l
bison mfpl.y
g++ mfpl.tab.c  -o mfpl_eval
mfpl_eval  inputFileName
```

Note that **we are no longer redirecting input using < in the command line**. This is necessary in order to be able to get input from the keyboard for the MFPL *(input)* statement at the time that it actually is evaluated. Consequently, you need to change your *main( )* function to process the input file from the command line:

```
int main(int argc, char** argv) {
  if (argc < 2) {
    printf("You must specify a file in the command line!\n");
    exit(1);
  }
  yyin = fopen(argv[1], "r");
  do {
        yyparse();
  } while (!feof(yyin));
  return 0;
}
```

As in HW #4, **no attempt should be made to recover from non-lexical errors**; if your program encounters a syntactical or semantic error, or **attempted**

division by zero (a new error you need to catch!), then it should simply output a meaningful message and terminate execution.

Your program should **still output the tokens and lexemes** that it encounters in the input file, and still should **output an appropriate message whenever it begins or ends a scope, or whenever it makes an entry in the symbol table**.

As before, your program should process a single expression from the input file (but remember that a single expression could be an expression list), terminating when it completes processing the expression or encounters an error. After reading in the expression from the input file, your program should evaluate and **output the resulting value of the expression**; this is known as the *read-eval-print* process of interpreters. Specifically, the N_START rule in your *bison* file should include the following output statements:

```
printRule("START", "EXPR");
printf("\n---- Completed parsing ----\n\n");
printf("\nValue of the expression is: ");
```

Note that because your program is now **evaluating** expressions in the input program, you also will need to **record an identifier's value in the symbol table**, which could be a string or an integer.

Programming Language

What follows is a brief description about the evaluation rules for the various expressions in MFPL (Mini Functional Programming Language).

<expr> → intconst | strconst | ident | (<parenthesized_expr>)

The resulting value of an **<expr>** is the value of the integer constant if the **intconst** rule is applied, or the value of the string constant if the **strconst** rule is applied, or the (string or integer) value of the identifier if the **ident** rule is applied, or the (string or integer) value of the **<parenthesized_expr>** if that rule is applied.

<parenthesized_expr> → <arithmetic_expr> | <if_expr> | <let_expr>
| <print_expr> | <input_expr> | <expr_list>

The resulting value of a **<parenthesized_expr>** is the (string or integer) value returned by whichever rule is applied. Note that **you are no longer required to process <lambda_expr>.**

<arithmetic_expr> → <op> <expr> <expr>

Depending on what **<op>** is, you will perform that operation on the values of the two **<expr>**'s. This will then be the resulting (integer) value for the **<arithmetic_expr>**. Be sure that you check to see if each **<expr>** is actually an

integer since input such as *(+ 5 (input))* is only valid if the user inputs an integer constant; if not, you should issue an appropriate error message. **Also note that if the <op> is division, you must check for attempted division by zero.**

<if_expr> → if <expr>$_1$ <expr>$_2$ <expr>$_3$

If **<expr>$_1$** evaluates to zero, then the resulting value of the **<if_expr>** is the (string or integer) value of **<expr>$_3$**; otherwise, the resulting value of the **<if_expr>** is the (string or integer) value of **<expr>$_2$**.

<let_expr> → let (<id_expr_list>) <expr>

<id_expr_list> → ε | <id_expr_list> (ident <expr>)

The value of each **ident** in **<id_expr_list>** will be the (string or integer) value of its **<expr>**. **This will need to be recorded in the symbol table!** Then the resulting value of the **<let_expr>** will be the (string or integer) value of its **<expr>**.

<lambda_expr> → lambda (<id_list>) <expr>

<id_list> → ε | <id_list> ident

For this assignment, your program will **NOT** have to handle functions.

<print_expr> → print <expr>

When a **<print_expr>** is processed, it should **output the (string or integer) value** of the **<expr>** followed by a newline to standard output. Also note that the resulting value of a **<print_expr>** is the (string or integer) value of the **<expr>**.

<input_expr> → input

When an **<input_expr>** is processed, it should perform a C or C++ *getline* call into a string (or char array); do not use *cin* >> since **valid input can contain spaces**. If the first character that is input is a digit or a '+' or a '–', treat the entire input as an integer; otherwise, treat the input as a string. Note that you should now set the type of an **<input_expr>** to either **INT** or **STR,** and dynamically type-check its use in other expressions.

<expr_list> → <expr> <expr_list> | <expr>

The resulting value of an **<expr_list>** is the (string or integer) value of the last **<expr>**.

What To Submit For Grading

You should submit via Blackboard any .h files necessary for your symbol table, as well as your *flex* and *bison* files. Do **NOT** submit a *make* file; your *flex* and *bison* files should *#include* your .h files as necessary. Name your *flex* and *bison* files using your last name followed by your first initial (e.g., Homer Simpson would name his files **simpsonh.l** and **simpsonh.y**).

Input Files

Sample input (and output) files are provided on the Blackboard website. You also are encouraged to create your own input files to test your program.

Note: Because we are using an automated script to grade your programs, and the output is fairly extensive, with the exception of whitespace and case (upper vs. lower) of the text, **your output is expected to EXACTLY MATCH the output specifications given in the homework assignments!** It is recommended that you use the *diff* command (see http://ss64.com/bash/diff.html) to compare your output with the sample output for this assignment.

Extra Credit

You can earn **extra credit** by making your program **evaluate *lambda* expressions**. The amount of extra credit will depend on the extent to which your program addresses this functionality (no pun intended ☺). A program that can only handle functions, the body of which is an arithmetic expression like ((lambda (x y) (+ x y)) 10 20), will not earn as many points as a program that can handle more complex expressions like ((lambda (x y) (let ((z (* x 10)) (foo (lambda (bar) (/ bar x)))) (- (foo z) y))) 10 20).

If you do the extra credit, **in addition** to submitting your files for "normal" grading through Blackboard, you must email Dr. Leopold (leopoldj@mst.edu):

(1) your flex, bison, and .h files,
(2) a brief, clear explanation of the extra credit capabilities of your program (e.g., my program can handle functions, the body of which can only be an arithmetic, print, or input expression), and
(3) sample input files that clearly and concisely demonstrate the extent of the extra credit capabilities of your program.

The extra credit can be submitted **any time before the last day of class this semester**. However, even if you do the extra credit, you are still required to submit the regular HW #5 by the specified due date!